

Agent Teams — feature spec

Status: proposal, ready to build · **Owner:** Harsha (@igharsha7) · **Author:** Nivesh (@nickthelegend) · **Date:** 2026-10-01
Branch: feat/agent-teams (off main, never directly on main)

1. The idea in one paragraph

Today Loom gives you **one agent per chat** and, for big goals, an **Orchestra**: an orchestrator plans a task graph, spawns a worker per task, and the workers die when their task merges. Loom Teams lets **several people** work on one repo without colliding. **Agent Teams joins the two**. Each person runs a **crew**: a named, long-lived group of agents with roles (a Lead, Builders, a Reviewer, a Tester). The crew keeps its context between goals, talks in a shared channel, claims work from the project board, reviews its own output across vendors, and ships **one PR per goal**. When several people each run a crew on the same repo, the crews coordinate through what Loom Teams already provides: presence, leases, the team brain and the landing queue. Humans stay in charge. You can talk to the whole crew or to any one teammate, and nothing merges without a human.

2. Why

What happens today	What it costs
An Orchestra worker starts cold for every task, and its context is thrown away when the task merges.	The next task re-reads the same files and repeats the same mistakes, and tokens go on rediscovery.
Workers never talk to each other; everything goes through the orchestrator.	The orchestrator is the bottleneck: a builder can't ask the tester "does this fail for you too?"
Review is one more agent turn, run when someone remembers.	Mixed-vendor output conflicts often. Our own research put it at 41.7% of overlapping agent pairs (docs/teams-architecture.md).
Each person's agents are separate from everyone else's, so a team of 5 is 5 people each driving 1–4 agents by hand.	No reusable, visible "who is doing what" for the agents, and the humans do the dispatching.

The pieces already exist: native continuity (persistent per-chat sessions across Codex, Claude Code and OpenCode), warm provider sessions, the board, Orchestra's worktrees and integration branch, and Loom Teams' hub, leases, brain and landing. **Agent Teams is mostly composition**, plus a small amount of new protocol.

3. Goals and non-goals

Goals

1. A person can create a crew from a template in under a minute and give it a goal or a backlog.
2. Teammates keep their own context across tasks and goals (native continuity per teammate).
3. Teammates coordinate directly, in a crew channel and through board cards, without routing everything through the Lead.
4. Built-in cross-vendor review and testing before anything reaches a PR.

5. With several people, crews never silently collide: leases, conflict prediction and the landing queue apply per teammate.
6. Everything is visible and steerable: the crew view, the channel, the board, the phone.
7. Every action is in the event log, replayable and auditable.

Non-goals (for now)

- Agents approving or merging PRs. A human always merges.
- Agents spending money (APIs, cloud) beyond the configured budgets.
- Cross-repo crews. A crew belongs to one project, just as the brain does.
- Replacing Orchestra. Orchestra becomes one *play* a Lead can run (§6.4).

4. Vocabulary

Term	Meaning
Crew	A named group of teammates attached to a project and owned by one person (e.g. "Nivesh's ship crew").
Teammate	One agent in a crew: a roster agent (codex, claude-code, opencode, a model agent...) plus a role and a charter .
Lead	The teammate that turns goals into cards, assigns them and integrates. Exactly one per crew.
Role	What a teammate is for: <code>lead</code> , <code>builder</code> , <code>reviewer</code> , <code>tester</code> , <code>researcher</code> , or a custom role. Roles set default permissions, briefing and which actions are allowed.
Charter	Standing instructions for a teammate (the existing per-agent instructions, scoped to the crew).
Crew channel	One chat where the crew and its humans talk. Every teammate also keeps its own chat (its private working thread).
Card	A board task: the unit of work a teammate claims. It carries <code>touches</code> , <code>blockedBy</code> , <code>priority</code> and <code>due</code> , all of which exist today.
Claim	A teammate taking a card. It takes the card's leases too (§7).
Handoff	A teammate passing a card to another teammate, with a note.
Play	A reusable way of working that the Lead can run: <i>build</i> → <i>review</i> → <i>test</i> → <i>land</i> , <i>race</i> (two builders, keep one), <i>orchestra</i> (DAG fan-out).

5. What the user sees

5.1 Creating a crew

Project settings → **Crews** → **New crew**, or `loom crew create --template ship`. The templates are:

Template	Teammates (defaults; every slot can be any installed agent)
Ship	Lead (Claude Code) · Builder ×2 (Codex, OpenCode) · Reviewer (a <i>different vendor</i> from the builders) · Tester (any)
Fix	Lead + Builder (same agent) · Tester
Research	Lead · Researcher ×2 · Writer
Solo+	One Builder + one Reviewer, no Lead (the human is the Lead)

The reviewer's vendor is checked on save. A crew whose reviewer shares a vendor with every builder gets a warning, not a block.

5.2 The Crew view

A new tab next to Orchestra, also shown as a sheet on the phone:

- **Roster:** each teammate's avatar, role, agent and model, live state (idle, working on card X, waiting, reviewing), context meter, and spend today.
- **Channel:** the crew chat. Messages from teammates are attributed (`@builder-1`), and humans can `@mention` any teammate or `@crew` .
- **Board lane:** the cards the crew owns, in its columns (Backlog → Claimed → In review → Testing → Ready to land), each with who holds it.
- **Goal strip:** the current goal, its integration branch, its PR, CI status and budget.

5.3 Talking to it

- Type in the channel: "add rate limiting to the API". The Lead answers with a plan (cards) before anyone starts. **Plan approval is on by default** for crews and can be turned off per crew.
- Type `@reviewer be strict about error handling` . That goes to one teammate and is added to its charter for this goal.
- Click any teammate to open its private thread, see exactly what it did, and steer it.
- **Stop crew** interrupts everyone. **Pause** stops new claims and lets current turns finish.

6. How work flows

6.1 One goal, one crew

1. A **goal** arrives: typed in the channel, picked from GitHub issues, or queued.
2. The **Lead plans**. It writes cards with `title` , `touches` (globs), `blockedBy` , and a suggested role. They go to the board and, if plan approval is on, wait for 👍 .
3. **Builders claim** ready cards, one card per builder at a time. Each builder works in its **own worktree** on `loom/crew/<crew>/<goal>/<card>` (reusing Orchestra's worktree plumbing) and posts progress to the channel.
4. When a builder finishes, the card moves to **In review**. The **Reviewer** gets the diff with the card and the plan, and either approves (the card moves to Testing) or **requests changes** (the card goes back to the same builder, *in the same session*, with the review attached).
5. The **Tester** runs the project's tests (`landing.fastTest` or detected scripts) on the card's branch. A failure goes back to the builder with the log tail.

6. The **Lead integrates** approved, green cards into the goal's integration branch (Orchestra's `integrate()`), resolves merge order, and opens **one PR per goal** (Loom Teams delivery, stacked when large).
7. The PR gets Loom Teams' required **cross-vendor review check** and a human CODEOWNER review. The **auto-fix loop** (two attempts) routes CI failures back to the builder that owns the files.
8. A human clicks **Land**.

6.2 Talking directly

Teammates can address each other in the channel without the Lead: "@tester does `npm test` fail on main too?", "@builder-2 I'm touching `src/api/*`, wait or narrow". A message to a busy teammate queues behind its current turn (the existing prompt queue). Messages are rate-limited per teammate so two agents can't loop forever (§10).

6.3 Continuity

Each teammate has its **own native session per crew** through native continuity (Harsha's Brain work). That session keeps the files, decisions and mistakes it has already seen, so the second card is cheaper and better than the first. A teammate moving between cards gets a *delta* packet (only what changed), not a full reconstruction. A crew-level memory scope (`scope: { crew }`) holds decisions the crew made, and it promotes to the project brain by the usual tiers.

6.4 Plays

The Lead chooses how to run each goal. It suggests a play, the human confirms, and the default is **Ship**:

Play	What it does	Built on
Ship	plan → claim → review → test → integrate → PR	new crew engine + Orchestra integrate
Race	two builders take the same card; the reviewer or the human keeps one	Orchestra race mode
Fan-out	a DAG of many small cards in parallel, ephemeral helpers allowed	Orchestra
Pair	builder and reviewer alternate on one card in a tight loop	crew engine

7. Several people, several crews ("both together")

Everything in Loom Teams applies, **per teammate instead of per person**:

- **Presence and Fleet**: teammates' crews appear in Fleet's Team section, grouped by owner. You see each crew's goal, cards and state, but not other people's transcripts (Loom Teams rule D21).
- **Leases**: a claim takes advisory leases on the card's `touches` , held by `member/crew/teammate` . Overlap rules are unchanged (D28–D39). A card that overlaps another member's lease is held with `overlap: wait | narrow | proceed:<reason>` before it starts. Hard zones from `loom.team.json` are enforced.
- **Conflict prediction**: `git merge-tree` against teammates' WIP refs runs on every card integration, and a predicted conflict notifies both crews' Leads *and* both humans.
- **Team brain**: crew decisions publish as proposals. Canon still changes only through the rolling `loom/canon` PR.

- **Asking another person's crew:** "@harsha/tester can you run the e2e suite on my branch?" is sent as a **request** that Harsha (the owner) accepts or declines on his phone. Crews never act for another member without that member's approval.
- **Runners:** a crew can be **hosted on a team runner** (Loom Teams Phase 5), so it keeps working while its owner's laptop is closed, under the runner's permission cap.
- **Policy** (`loom.team.json` , read from `origin/HEAD`), with these new fields:

```
"crews": {
  "maxPerMember": 1,
  "maxTeammates": 6,
  "reviewerMustDifferFromBuilders": true,
  "planApproval": "required",
  "budgets": { "perCrewDailyUsd": 20 },
  "roles": { "builder": { "permissions": "auto" }, "reviewer": { "permissions": "ask" } }
}
```

- **Budgets:** spend rolls up per teammate, per crew, per member and per goal (the turn-stats and feed costs exist). A crew over budget pauses and asks.

8. Architecture

8.1 New pieces

Piece	Where	What it does
CrewEngine	<code>src/core/crew.ts</code>	Per-project owner of crews: the scheduler (who takes the next turn), card claims and handoffs, review/test transitions, and plays. Pure logic over a host interface, like <code>OrchestraEngine</code> .
Crew protocol	<code>src/core/crew-protocol.ts</code>	The actions teammates emit in a <code>loom</code> code block, the same way the orchestrator does today, so any agent can take part even with no tools. A parser plus validation per role.
Crew routes	<code>src/daemon/routes/crews.ts</code>	REST for crews, the roster, the channel, claims and plays, plus WebSocket frames (<code>crew</code>).
Crew view	<code>src/web/project/crew.js</code>	The tab in §5.2, as a project factory like the others. Phone: <code>app/src/crew.tsx</code> .
CLI	<code>loom crew</code> <code>create/status/say/assign/pause/stop</code>	Thin over the routes.

8.2 Reused pieces (no new copies)

- **Agents and sessions:** roster agents and adapters; warm provider sessions; **native continuity** gives each teammate a binding per (crew, teammate).
- **Board:** cards gain `crew` , `claimedBy` , `stage` (claimed | review | test | ready) and `reviews[]` . They stay one board, one store.
- **Worktrees, integration, race:** from `OrchestraEngine` . Lift what's needed into a shared `src/core/worktrees.ts` instead of duplicating it.
- **Prompt queue:** a message to a busy teammate queues in that teammate's lane.

- **Leases, policy, brain, landing:** Loom Teams modules, called with a teammate as the holder.
- **Approvals and permissions:** per-role defaults through the existing permission modes.

8.3 The protocol (first cut)

```
{
  "actions": [
    {
      "type": "plan",
      "cards": [
        {
          "title": "Rate-limit /api",
          "touches": ["src/api/**"],
          "role": "builder",
          "blockedBy": []
        }
      ]
    },
    {
      "type": "claim",
      "card": "c12"
    },
    {
      "type": "post",
      "to": "@tester",
      "text": "does npm test fail on main too?"
    },
    {
      "type": "handoff",
      "card": "c12",
      "to": "@builder-2",
      "note": "needs the cache layer first"
    },
    {
      "type": "review",
      "card": "c12",
      "verdict": "approve",
      "notes": ""
    },
    {
      "type": "test",
      "card": "c12",
      "result": "fail",
      "log": "...tail..."
    },
    {
      "type": "ask",
      "question": "Postgres or SQLite for the limiter store?"
    },
    {
      "type": "done",
      "summary": ""
    }
  ]
}
```

Allowed actions per role: the Lead may `plan`, `handoff`, `ask` and `done`; a builder may `claim`, `post`, `handoff` and `ask`; the reviewer may `review` and `post`; the tester may `test` and `post`. Anything else is refused and the refusal is logged as a reply. Later, the same actions are offered as **MCP tools** for agents that support MCP, which removes the parsing step.

8.4 Data

- **Config** (`.loom/config.json`):

```
{
  "crews": [
    {
      "id": "ship",
      "name": "Ship crew",
      "play": "ship",
      "planApproval": true,
      "teammates": [
        {
          "id": "lead",
          "agent": "claude-code",
          "role": "lead"
        },
        {
          "id": "builder-1",
          "agent": "codex",
          "role": "builder"
        },
        {
          "id": "builder-2",
          "agent": "opencode",
          "role": "builder"
        },
        {
          "id": "reviewer",
          "agent": "codex",
          "role": "reviewer",
          "charter": "be strict about errors"
        },
        {
          "id": "tester",
          "agent": "opencode",
          "role": "tester"
        }
      ]
    }
  ]
}
```

A teammate references a roster agent. Several teammates can share one roster agent kind, each with its own session.

- **State:** crew runtime state (current goal, claims, stages) lives in `.loom/crews/<id>.json`, written like orchestra runs.
- **Events:** new kinds `crew` (phase changes) and `crew_message` (channel posts). Everything is in the log, so the UI, the phone and replay all read the same thing.
- **Hub** (several people only): `presence` rows gain `crew` and `teammate` columns. Leases already carry free-form holder metadata. No new tables are needed for Phase 4, but if one is, it needs a migration and **the owner's approval before it's applied to the hosted Supabase**.

8.5 The scheduler

- At most one turn at a time per teammate. Per-crew concurrency is capped (`maxParallel`, default 3).
- **Priority:** human messages first, then review/test (to unblock builders), then claims by card `priority` and `due`.
- **Loop guards:** a teammate↔teammate exchange is capped at N messages per card (default 6) before it asks the Lead. The Lead's handoffs per card are capped at 3 before it asks the human.

- Stop and pause propagate to every teammate. Restart **resumes** crews the way Orchestra runs resume (interrupted turns continue, and the Lead is re-briefed).

9. API and CLI

Endpoint	Purpose	
GET/POST /api/projects/:id/crews	List and create crews (from a template).	
GET/PATCH/DELETE /api/projects/:id/crews/:crew	Read, edit and remove a crew.	
POST /api/projects/:id/crews/:crew/say	Post to the channel ({text, to?}).	
POST /api/projects/:id/crews/:crew/goal	Start a goal ({text \	issue, play?}).
POST /api/projects/:id/crews/:crew/{pause,resume,stop}	Control.	
POST /api/projects/:id/crews/:crew/cards/:card/{approve,reassign,unclaim}	Human overrides.	
WS frame crew	Roster and stage changes, live.	

CLI: loom crew create --template ship, loom crew status, loom crew say "@reviewer ...", loom crew goal "add rate limiting", loom crew pause|stop.

10. Risks and how we handle them

Risk	Mitigation
Agents chatting in loops and burning tokens	Message caps per card, budgets per crew, and a "needs you" escalation; the scheduler refuses a teammate replying to itself.
Same-vendor rubber-stamp reviews	The reviewer's vendor must differ (policy), and reviews must cite file:line.
Parallel builders colliding inside one crew	Card touches and leases inside the crew too, plus merge-tree prediction on every integration.
Cost surprise	Per-teammate spend live in the roster, daily caps, and plan approval on by default.
Context bloat in long-lived sessions	Native continuity's delta packets and compaction handling; the Lead can refresh a teammate (new epoch).
Prompt injection from issues and web content	Memories from untrusted turns stay personal (Loom Teams D45); builders can't change crew policy.
One flaky teammate blocks the crew	Health checks (the existing agent Check), reassign on repeated failure, and a fallback agent per role.

11. Build plan (for Harsha)

Work on `feat/agent-teams` off `main`. **Don't push to `main`**. Open one PR per phase against `main`. Each PR must pass `npm test` (all DOM suites included) and CI.

Phase	Scope	Done when
P1 — Crew core (one person, sequential)	config, <code>CrewEngine</code> , protocol parser, the channel and card claims, the Ship play with review/test stages, routes, CLI, a minimal Crew tab	a crew of fake agents (echo plus scripted adapters) plans, claims, reviews, tests and integrates a goal end to end in tests; a live run with OpenCode + Codex works
P2 — Persistent teammates	a native continuity binding per teammate, crew memory scope, refresh/epoch, resume after restart	the second card on the same teammate gets a delta packet; a daemon restart mid-goal resumes it
P3 — Parallel and safe	worktrees per builder, in-crew leases, merge-tree prediction, the Race and Fan-out plays (reusing Orchestra)	two builders on overlapping cards are held and resolved; integration conflicts route back to the owner
P4 — Several people	crews in hub presence and Fleet, cross-member requests with owner approval, runner-hosted crews, <code>loom.team.json</code> crew policy, budgets	two members' crews on one repo (the existing two-member test harness) never write the same file without a hold
P5 — Polish	templates, phone Crew sheet, notifications, the Crew view's board lane, docs	the phone can start a goal, approve a plan, and watch cards move

Testing rules

- Never drive the real Claude Code in tests. Use echo, scripted adapters and the native fakes (`test/native-fakes.ts`, `test/opencode-fake.ts`).
- A live check uses OpenCode's free model (`opencode/big-pickle`) and, sparingly, Codex.
- Every new route gets an HTTP test; every UI change gets a DOM test (the `src/web` checkJs and accessor checks must stay at zero).

Ask before: applying any hosted Supabase migration, changing landing or merge behaviour, or adding a paid dependency.

12. Open questions

1. Should a crew be able to **own several goals at once**, or one goal at a time in v1? (Proposal: one at a time.)
2. Should teammates get **MCP tools** in P1, or the `loom` code block only? (Proposal: block first, MCP in P3.)
3. **Lending:** can a member lend one teammate (say, a reviewer) to another member's crew for a goal? (Proposal: P4, owner-approved.)
4. Pricing: does a crew teammate count toward seats? (Proposal: no. Seats are humans; budgets cap agents.)

13. Pointers

- Loom Teams design and decisions D1–D78: `docs/teams-architecture.md`, `docs/teams.md`
- Native continuity: `docs/brain-continuity.md`, `docs/refactoring/BRAIN-TODO.md`
- Orchestra: `src/core/orchestra.ts` (plan protocol, worktrees, integrate, race, resume)

- Provider core and warm sessions: `src/providers/` , `docs/refactoring/T3-PORT-NOTES.md`
- Web client structure: `src/web/README.md`